# CLINISCAN PROJECT - DEEP LEARNING FOR MEDICAL IMAGING

## Milestone 4

**Final Evaluation, Documentation & Deployment Report**

**CliniScan - Chest X-ray Abnormality Detection System**

**Author:** Adithya Jayaram

**Year:** 2026

### Project Overview

**Dataset:** VinDr-CXR (15,000 images)

**Framework:** Streamlit (Unified Frontend & Backend)

**Core Models:** ResNet18 (PyTorch) + YOLOv8s (Ultralytics)

### Web Application Development

- [x] Unified Web Application deployed on Streamlit Cloud
- [x] Binary Classification (Normal vs. Abnormal) via ResNet18
- [x] Localized Object Detection via YOLOv8s
- [x] Visual Explainability via Grad-CAM
- [x] Automated PDF Report Generation
- [x] Real-time inference pipeline integration

**Live Application:** https://b13-cliniscan-adithya-jayaram-ppgjlzg9mpndfqyschjatz.streamlit.app

**Repository:** https://github.com/GKSJ-AI-CliniScan/B13-CliniScan/tree/Adithya-Jayaram

# TABLE OF CONTENTS

# PART A

## PROJECT OVERVIEW & MILESTONE OBJECTIVES

### Milestone 4 Objective

Milestone 4 represents the final phase of the CliniScan project, transforming the trained AI models (ResNet18 and YOLOv8s) into a fully functional, publicly accessible web application. This milestone encompasses pipeline integration, explainability features, user interface design, and production deployment.

| Objective | Implementation | Status |
|---|---|---|
| **Application Server** | Streamlit unified application handling both UI and inference | ✓ Complete |
| **Model Serving** | ResNet18 (Classification) + YOLOv8s (Detection) loaded via PyTorch/Ultralytics | ✓ Complete |
| **Explainability** | Grad-CAM implementation for ResNet18 to highlight abnormal regions | ✓ Complete |
| **User Interface** | Python-based Streamlit UI with interactive elements | ✓ Complete |
| **Image Analysis** | Two-stage pipeline: Classification -> Detection (if abnormal) | ✓ Complete |
| **Reporting** | Automated PDF report generation containing diagnostic results | ✓ Complete |
| **Cloud Deployment** | Production deployment on Streamlit Cloud | ✓ Complete |

# PART B

## SYSTEM ARCHITECTURE

### B.1 Full-Stack Architecture Overview

CliniScan utilizes a streamlined, monolithic architecture powered by Streamlit. Instead of decoupled frontend and backend servers, Streamlit handles user interactions and executes backend Python logic within the same unified environment, ensuring low latency and seamless data passing.

User Browser ➔ Streamlit Cloud Container

- **UI Layer:** Streamlit widgets (File Uploader, Buttons, Image Display)
- **Logic Layer:** State management, pre-processing, PDF generator
- **Inference Layer:** ResNet18 (Classifier) ➔ YOLOv8s (Detector) ➔ Grad-CAM

### B.2 Core Processing Architecture

The application operates inside a containerized environment on Streamlit Cloud. It loads both AI models into memory upon startup and executes a two-stage inference pipeline based on user input.

| Component | Technology | Purpose |
|---|---|---|
| **Web Framework** | Streamlit | Unified UI rendering and routing logic |
| **Classification** | PyTorch + ResNet18 | Binary classification (Normal vs Abnormal) |
| **Detection** | Ultralytics YOLOv8s | Bounding box localization for abnormal cases |
| **Interpretability** | Custom Grad-CAM | Gradient-weighted activation map generation |
| **Data Processing** | OpenCV / Pillow | Image resizing, normalization, and augmentation |

| | | |
|---|---|---|
| **Deployment** | Streamlit Cloud | Public hosting linked directly to GitHub repository |

# PART C

## CORE INFERENCE PIPELINE (BACKEND LOGIC)

### C.1 Technology Stack

| Package | Role |
|---|---|
| streamlit | Core web framework and state management |
| torch + torchvision | Deep learning framework + ResNet18 architecture |
| ultralytics | YOLOv8s model loading and inference |
| opencv-python-headless | Image processing and bounding box annotations |
| Pillow | Image loading and format conversion |
| fpdf / reportlab | PDF report generation logic |

### C.2 Pipeline Execution Flow

| Stage | Input | Action | Output |
|---|---|---|---|
| **1. Upload** | Image File | Validate and convert to required tensor shape | Processed Image Tensor |
| **2. Classify** | Image Tensor | Pass through ResNet18 | "Normal" or "Abnormal" |
| **3. Detect** | Original Image | If "Abnormal", pass | Bounding Box |

| | | | |
|---|---|---|---|
| | | to YOLOv8s | Coordinates |
| **4. Explain** | Image Tensor | Extract gradients from ResNet18 | Heatmap Overlay |

### C.3 Model Integration

Models are loaded efficiently using Streamlit's caching mechanisms to prevent reloading weights on every user interaction.

```
# Cached Classification model loading
@st.cache_resource
def load_classifier():
    model = resnet18(weights=None)
    model.fc = nn.Linear(model.fc.in_features, 2) # Binary classification
    model.load_state_dict(torch.load('weights/resnet18_best.pth', map_location='cpu'))
    model.eval()
    return model

# Cached Detection model loading
@st.cache_resource
def load_detector():
    return YOLO('weights/yolov8s_best.pt')
```

### C.4 Grad-CAM Implementation

Gradient-weighted Class Activation Mapping is implemented to visualize which regions of the X-ray heavily influenced the ResNet18 model's prediction.

```
# Hook registration on ResNet18 final convolutional layer
target_layer.register_forward_hook(save_activation)
target_layer.register_backward_hook(save_gradient)

# Heatmap Generation Process
# 1. Calculate weights from gradients
# 2. Apply weights to activations
# 3. Apply ReLU to retain only positive features
# 4. Normalize and apply colormap (cv2.COLORMAP_JET) overlay
```

# PART D

## USER INTERFACE (FRONTEND DEVELOPMENT)

### D.1 Technology Stack

| Technology | Purpose |
|---|---|
| **Streamlit UI** | Main layout, sidebars, interactive widgets, and layout columns |
| **Markdown/HTML** | Custom styling, text formatting, and titles within the Streamlit app |
| **Matplotlib / Altair** | Rendering data visualizations and performance metrics |

### D.2 Core Features

- **X-ray Upload Zone:** An interactive drag-and-drop file uploader accepting standard image formats (PNG, JPG, JPEG). Previews the uploaded image on the screen.
- **Two-Stage Analysis Trigger:** A unified action button that initiates the deep learning pipeline, managing loading states automatically.
- **Classification Results Panel:** Displays the ResNet18 binary prediction clearly (Normal vs. Abnormal) using color-coded text and confidence scores.
- **Detection Results Panel (Conditional):** If an abnormality is detected, the UI renders the X-ray with YOLOv8s bounding box annotations directly overlaid, indicating specific lesions.
- **Grad-CAM Visualization:** A side-by-side comparison showing the original X-ray alongside the Grad-CAM heatmap, providing a visual explanation for the clinician.
- **PDF Report Generator:** A custom module that compiles the patient image, classification result, detection bounding boxes, and Grad-CAM heatmap into a downloadable PDF document.

# PART E

## APPLICATION INTERFACE - SCREENSHOTS

*(Note: Replace these placeholders with actual screenshots of your Streamlit application)*

**Screenshot 1 — Main Dashboard & Image Upload**

*(Insert image showing the Streamlit title, sidebar, and the file upload drag-and-drop zone)*

**Description:** The main landing interface where users can upload Chest X-rays for analysis.

**Screenshot 2 — Classification & Detection Results**

*(Insert image showing the predicted class and bounding boxes)*

**Description:** Application displaying a positive "Abnormal" prediction alongside the YOLOv8s output highlighting the localized abnormality.

**Screenshot 3 — Grad-CAM Explainability**

*(Insert image showing the heatmap overlay)*

**Description:** Visual explanation panel showing the Grad-CAM heatmap, highlighting the areas the ResNet18 model focused on.

**Screenshot 4 — PDF Report Download**

*(Insert image showing the download button and a sample PDF)*

**Description:** The final report generation interface allowing users to download a comprehensive diagnostic summary.

# PART F

## DEPLOYMENT

### F.1 Streamlit Cloud Deployment

The CliniScan application is deployed via Streamlit Community Cloud. This platform provides continuous integration directly from the project's GitHub repository, automatically rebuilding the application whenever new code is pushed.

| Configuration | Value |
|---|---|
| **Platform** | Streamlit Community Cloud |
| **Live URL** | *(Paste your Streamlit Cloud URL here)* |
| **GitHub Repo** | *(Paste your GitHub repo URL here)* |
| **Main Entry Point** | app.py |
| **Model Weights** | Stored locally in repo or via external cloud storage (e.g., Google Drive / Hugging Face Hub) depending on size |
| **Auto Deploy** | Enabled – updates on every push to the main branch |

### F.2 Environment Configuration

A standard requirements.txt is utilized to configure the Streamlit container environment.

```
# requirements.txt snippet
streamlit==1.32.0
torch==2.1.0
torchvision==0.16.0
ultralytics==8.1.0
opencv-python-headless==4.9.0
Pillow==10.2.0
```

# PART G

## EXPERIMENTS & ERROR ANALYSIS

Because Streamlit operates as a single internal pipeline, testing focused on the integration of the models, hyperparameter optimization, and analyzing edge cases within the pipeline workflow.

### G.1 Hyperparameter Tuning & Augmentation

Several experiments were conducted to optimize model performance before integrating them into the final deployment:

- **Learning Rate Adjustments:** Adjusted from 0.0003 to 0.0001, yielding minor performance improvements and better convergence stability.
- **Data Augmentation Strategy:** Applied flips, rotations, brightness adjustments, and CLAHE (Contrast Limited Adaptive Histogram Equalization).
    - *Outcome:* Improved recall for abnormal cases, but slightly reduced overall accuracy.
- **Alternative Architectures:** Tested DenseNet121 via transfer learning.
    - *Outcome:* Showed lower performance due to specific dataset class imbalances compared to the chosen ResNet18 architecture.

### G.2 Error Analysis

Reviewing the pipeline output highlighted key challenges in automated medical imaging analysis:

| Error Type | Observation | Cause / Context |
|---|---|---|
| **False Positives** | Rib shadows misclassified | Normal anatomical structures mimicking the density of pathological abnormalities. |
| **False Negatives** | Small lesions bypassed | Subtle or highly localized patterns not captured efficiently by the global classification weights. |

*Note: This highlights the inherent challenge of detecting subtle patterns in complex medical imaging and justifies the inclusion of the YOLOv8s detection model to force localized evaluation.*

# PART H

## DELIVERABLES & PROJECT COMPLETION

**Milestone 4 Deliverables Checklist**

- [x] **Milestone 4 PDF Report** - Theory, architecture, implementation, and deployment details.
- [x] **Application Source Code** - app.py, models, and UI logic.
- [x] **Deployment Files** - requirements.txt configuration.
- [x] **Live Application URL** - *(Your Streamlit URL)*
- [x] **Classification Module** - ResNet18 integration.
- [x] **Detection Module** - YOLOv8s integration.
- [x] **Explainability Module** - Grad-CAM integration.
- [x] **PDF Report Generation** - Automated summary generation.

**Complete Project Milestone Summary**

| Milestone | Focus Area | Key Deliverable | Status |
|---|---|---|---|
| **Milestone 1** | Data & Preprocessing | 15,000 VinDr-CXR images prepared, Train/Test splits established | ✓ Done |
| **Milestone 2** | Baseline Model Training | Initial binary classification setup and baseline bounding boxes | ✓ Done |
| **Milestone 3** | Model Optimization | Trained ResNet18 and YOLOv8s with hyperparameter tuning | ✓ Done |
| **Milestone 4** | App UI + Deployment | Streamlit web application and cloud hosting | ✓ Done |

# PART I

## CONCLUSION & LIMITATIONS

### Conclusion

Milestone 4 successfully demonstrates the application of deep learning in medical imaging through a complete, end-to-end pipeline. The CliniScan system is now fully deployed and accessible via a web interface, transforming the trained data models into a practical decision-support tool.

The integration of ResNet18 for classification, YOLOv8s for localized detection, and Grad-CAM for visual explainability provides a comprehensive approach to automated X-ray analysis. The deployment via Streamlit Cloud highlights its practical usability, user-friendliness, and potential for further clinical development.

### Final Model Performance Summary

| Model | Task | Metric | Score |
|---|---|---|---|
| **ResNet18** | Classification | Accuracy | 70% |
| **ResNet18** | Classification | ROC-AUC | 0.966 |
| **ResNet18** | Classification | F1-score (Abnormal) | 0.73 |
| **YOLOv8s** | Detection | Precision | 0.5459 |
| **YOLOv8s** | Detection | Recall | 0.3703 |
| **YOLOv8s** | Detection | mAP@50 | 0.4132 |
| **YOLOv8s** | Detection | mAP@50-95 | 0.2248 |

### Limitations & Future Work

While the classifier shows strong performance (ROC-AUC 0.96), object detection in medical contexts remains highly challenging, reflected in the moderate mAP scores. Current limitations include binary-only classification, a constrained dataset size, and a lack of formal clinical validation.

Future work will focus on expanding to multi-class classification (identifying specific diseases), improving detection accuracy with larger datasets, conducting clinical validation studies, and establishing secure integrations into existing hospital data systems.

**CliniScan AI Live Application:**
*https://b13-cliniscan-adithya-jayaram-ppgjlzg9mpndfqyschjatz.streamlit.app*

## Interface Demo: